



**UNIVERSITAT POLITÈCNICA DE CATALUNYA  
BARCELONATECH**

---

**Escola Superior d'Enginyeries Industrial,  
Aeroespacial i Audiovisual de Terrassa**

Bachelor's Degree in Aerospace Vehicle Engineering

---

## **Simple 1D steady-state case of conduction heat transfer in a composite wall**

---

**Gas dynamics and heat and mass transfer**  
Marina Liao Colom Escolà

October 7, 2025

## Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Studied case description</b>                  | <b>2</b>  |
| <b>2</b> | <b>Analytical resolution</b>                     | <b>3</b>  |
| <b>3</b> | <b>Numerical resolution: Gauss-Seidel method</b> | <b>4</b>  |
| <b>4</b> | <b>Implemented code</b>                          | <b>6</b>  |
| <b>5</b> | <b>Conclusions</b>                               | <b>11</b> |

# 1 Studied case description

Through this practice, a simple case of 1D conduction heat transfer in steady-state has been studied. Specifically, a wall of total length  $L$  composed of two materials joined at  $x_{change}$  (Figure 1) has been analyzed.

The first segment ( $0 < x < x_{change}$ ) is made of material A with thermal conductivity  $\lambda_1$ , and the second one ( $x_{change} < x < L$ ) is made of material B with thermal conductivity  $\lambda_2$ . The system is in steady-state, and heat conduction is one-dimensional along the x-axis. The temperature at the left end of the wall is maintained at  $T_0$  and at the right end, it is maintained at  $T_L$ .

There is no internal heat generation and no convection heat transfer. It also can be assumed a perfect thermal contact between the two materials and a constant thermal conductivity in each material.

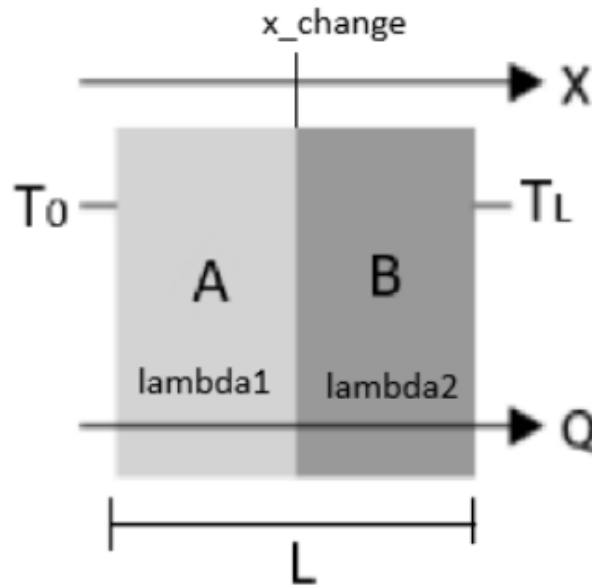


Figure 1: Case sketch

## 2 Analytical resolution

This problem can be solved analytically. As it is stationary ( $\delta t = 0$ ), the variation of the internal energy is zero ( $dU = 0$ ), and assuming constant physical properties, the Laplace's equation for the field of temperatures in conduction can be expressed as:

$$\nabla^2 T = 0$$

Hence, for one-dimension:

$$\frac{\delta^2 T}{\delta x^2} = 0$$

And its solution has the following form:

$$T(x) = ax + b$$

This solution can be applied separately in both segments of the wall with its corresponding boundary conditions.

$$T_A(x) = C_1x + C_2$$

$$T_B(x) = C_3x + C_4$$

$$\begin{cases} x = 0 : & T(0) = T_0 = C_2 \\ x = x_{change} : & T(x_{change}) = C_1x_{change} + C_2 \\ x = x_{change} : & T(x_{change}) = C_3x_{change} + C_4 \\ x = L : & T(L) = T_L = C_3L + C_4 \end{cases}$$

$$C_1 = \frac{T(x_{change}) - T_0}{x_{change}} \quad C_2 = T_0$$

$$C_3 = \frac{T_L - T(x_{change})}{L - x_{change}} \quad C_4 = \frac{LT(x_{change}) - x_{change}T_L}{L - x_{change}}$$

Then, the heat flow for both segments is:

$$q(x) = -\lambda \frac{dT}{dx}$$

$$q_A(x) = -\lambda_1 C_1 = -\lambda_1 \frac{T(x_{change}) - T_0}{x_{change}}$$

$$q_B(x) = -\lambda_2 C_1 = -\lambda_2 \frac{T_L - T(x_{change})}{L - x_{change}}$$

And  $T(x_{change})$  can be calculated due to the continuity of the heat flow.

$$\begin{aligned} q_A(x_{change}) &= q_B(x_{change}) \\ -\lambda_1 \frac{T(x_{change}) - T_0}{x_{change}} &= -\lambda_2 \frac{T_L - T(x_{change})}{L - x_{change}} \\ T(x_{change}) &= \frac{\lambda_1(L - x_{change})T_0 + \lambda_2x_{change}T_L}{\lambda_1(L - x_{change}) + \lambda_2x_{change}} \end{aligned}$$

Finally, its field of temperatures and its heat flow are expressed as:

$$T(x) = \begin{cases} T_0 + \frac{\lambda_2(T_L - T_0)}{\lambda_1(L - x_{change}) + \lambda_2x_{change}} x, & 0 \leq x \leq x_{change}, \\ \frac{\lambda_1(L - x_{change})T_0 + \lambda_2x_{change}T_L}{\lambda_1(L - x_{change}) + \lambda_2x_{change}} + \frac{\lambda_1(T_L - T_0)}{\lambda_1(L - x_{change}) + \lambda_2x_{change}} (x - x_{change}), & x_{change} \leq x \leq L. \end{cases}$$

$$q = -\frac{\lambda_1\lambda_2(T_L - T_0)}{\lambda_1(L - x_{change}) + \lambda_2x_{change}}$$

### 3 Numerical resolution: Gauss-Seidel method

For the numerical resolution of this problem, the Gauss-Seidel method has been used. Also, it is applied in the code solver shown in next section.

The domain can be discretized in  $N$  control volumes. By making an uniform mesh,  $N$  nodes can be located in the center of each control volume at a given distance:

$$\Delta x = \frac{L}{N}$$

Also, two external nodes are defined at the beginning ( $x = 0$ ) and end ( $x = L$ ) of the wall, having a total of  $N + 2$  nodes. Thus, the position along the x-axis of the control volumes  $x_{VC}[i]$  and the nodes  $x[i]$  can be defined as:

$$\begin{aligned} x_{VC}[i] &= (i - 1)\Delta x & i &= [1, N + 1] \\ x[i] &= \frac{x_{VC}[i] + x_{VC}[i - 1]}{2} & i &= [2, N + 1] \\ x[1] &= 0 & x[N + 2] &= L \end{aligned}$$

Thereafter, it is determined the equation of discretization of the inner and outer nodes. From the energy balance, it is obtained:

$$\begin{aligned} \dot{Q} &= 0 \\ \dot{Q}_w - \dot{Q}_e &= 0 \\ -\lambda_w \frac{T_P - T_W}{d_{PW}} S_w + \lambda_e \frac{T_E - T_P}{d_{PE}} S_e &= 0 \end{aligned}$$

where  $W$  is the left node of the current node  $P$ , and  $E$ , its right one. Accordingly, the discretized equation for internal nodes is:

$$a_P T_P = a_E T_E + a_W T_W + b_P$$

being

$$\begin{aligned} a_W &= \lambda_w S_w / d_{PW} \\ a_E &= \lambda_e S_e / d_{PE} \\ a_P &= a_E + a_W \\ b_P &= 0 \end{aligned}$$

In the same way, the coefficients of this equation for the external nodes can be deduced:

$$\begin{aligned} i = 1 & \begin{cases} a_W = 0 \\ a_E = \lambda_e S_e / d_{PE} \\ a_P = a_E \\ b_P = 0 \end{cases} \\ i = N + 2 & \begin{cases} a_W = \lambda_w S_w / d_{PW} \\ a_E = 0 \\ a_P = a_W \\ b_P = 0 \end{cases} \end{aligned}$$

As the wall is made of two materials, these  $\lambda$ 's are the harmonic thermal conductivities. For an uniform mesh (same distances between nodes and control volumes), it can be written as:

$$\lambda_{harm} = \frac{2\lambda_i \lambda_j}{\lambda_i + \lambda_j}$$

All surfaces and distances can be simplified from the equations due to this uniformity of the mesh on which it has been worked.

The algorithm for the numerical resolution is described as:

1. Data entry: physical  $(L, \lambda_1, \lambda_2, \dots)$  and numerical  $(N, \delta, T_{start})$  known data.
2. Definition of vectors and previous calculations: e.g.  $x[i]$ ,  $x_{VC}[i]$ .
3. Initial map of temperatures  $T^*[i] = T_{start}$  and boundary conditions  $T[1] = T_0$ ;  $T[N + 2] = T_L$ .
4. Calculation of the coefficients:  $a_P[i]$ ,  $a_W[i]$ ,  $a_E[i]$ ,  $b_P[i]$ .
5. Gauss-Seidel solver:

$$T[i] = \frac{a_E[i]T[i + 1] + a_W[i]T[i - 1] + b_P[i]}{a_P[i]}$$

6. Is  $\max|T[i] - T^*[i]| < \delta?$ .  
 NO:  $T^*[i] = T[i]$  and point 5.  
 YES: point 6.

7. Final calculations and print results.

8. End.

```
while(1) iter++; GaussSeidel(N, aW, aP, aE, Tc, Ts, maxerror); if(maxerror < delta)break;
```

## 4 Implemented code

The following code in C has implemented the algorithm described in the previous page for the resolution of the problem studied. It has to be remarked that in C the index count starts from 0, so the nodes numbering goes from 0 to N+1, and not as the previous section that it was described as 1 to N+2.

The code is divided into the main code "main.c" and the .c files located in "sources\" where the different used functions are defined. The corresponding headers are in "headers\" in .h format. Required standard libraries are: stdlib.h, stdio.h and math.h. And, it is recommended to run and compile with the following commands using gcc:

```
1 gcc .\main.c .\sources\mesh.c .\sources\print.c .\sources\solver.c -I .\headers -o main
```

```
1 ./main
```

Next lines show the implemented code file by file.

main.c

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <math.h>
4
5 #include "headers/print.h"
6 #include "headers/mesh.h"
7 #include "headers/solver.h"
8
9 int main(){
10     //1. ENTRADA DE DADES
11     //fisiques
12     const double L = 5.0, xchange = 2.8;
13     const double lambda1 = 120, lambda2 = 180;
14     const double T0 = 100, TL = 200;
15     //numeriques
16     const int N = 100; const double delta = 1e-9, Tstart = (T0+TL)/2;
17
18     //2. DEFINICIO DE VECTORS I CALCULS PREVIS
19     double *x = (double*)malloc((N+2)*sizeof(double)); //posicio dels nodes
20     double *x_vc = (double*)malloc((N+1)*sizeof(double)); //posicio de les cares
21     double *lambdas = (double*)malloc((N+2)*sizeof(double)); //lambdas dels nodes
22     const double dx = L/N; //distancia entre cada node
23     generate_mesh(x, dx, xchange, N, L, lambdas, lambda1, lambda2, x_vc);
24
25     //3. MAPA INICIAL DE TEMPERATURA
26     double *T_s = (double*)malloc((N+2)*sizeof(double)); //temperatura suposada
27     double *T_c = (double*)malloc((N+2)*sizeof(double)); //temperatura calculada
28
29     for(int i=0; i<N+2; i++)
30         T_s[i] = Tstart;
31     T_s[0]=T0; T_s[N+1]=TL;
32
33     //4. CALCUL DE COEFICIENTS DE DISCRETITZACIO
34     double *aE = (double*)malloc((N+2)*sizeof(double));
35     double *aP = (double*)malloc((N+2)*sizeof(double));
36     double *aW = (double*)malloc((N+2)*sizeof(double));
37     //bP = 0 (no hi ha fonts internes ni conveccio)
38
39     coeff(N, aW, aP, aE, lambdas);
40
41     //5. SOLVER (GAUSS-SEIDEL)
42     int iter=0; double max_error;
43     while(1){
44         iter++;
45         GaussSeidel( N, aW, aP, aE, T_c, T_s, &max_error);
46         if(max_error<delta) //criteri de convergencia
47             break;
```

```

48     for(int i=0;i<N+2;i++)
49         T_s[i]=T_c[i]; //actualitzar
50     }
51
52     free(T_s); free(aE); free(aW); free(aP);
53
54     //7.CALCULS FINALS I IMPRESSIO DE RESULTATS
55     print_state(L,xchange,lambda1,lambda2,T0,TL);
56     print_mesh(N, dx, x, lambdas, x_vc);
57     print_initialf(Tstart, T0, TL);
58     print_coeff();
59     print_solver();
60     print_results(delta, iter, N, T_c);
61
62     free(T_c); free(x); free(x_vc); free(lambdas);
63
64     //8.FI
65     return 0;
66 }

```

sources\mesh.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include "../headers/mesh.h"
4
5  void generate_mesh(double *x, double dx, double xchange,
6                    int N, double L, double *lambdas,
7                    double lambda1, double lambda2,
8                    double *x_vc){
9
10     x[0]=0; x[N+1]=L;
11     x_vc[0]=0; lambdas[0]=lambda1; lambdas[N+1]=lambda2;
12     for(int i=1;i<N+1;i++){
13         x_vc[i]=dx*i;
14         x[i]=x_vc[i]-dx/2;
15         if(x_vc[i]<xchange)
16             lambdas[i]=lambda1;
17         else
18             lambdas[i]=lambda2;
19     }
20 }

```

sources\solver.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <math.h>
4  #include "../headers/solver.h"
5
6  double lambda_arm(double lambdai, double lambdaj){
7     return ((2*lambdai*lambdaj)/(lambdai+lambdaj));
8 }
9
10 void coeff(int N, double *aW, double *aP, double *aE, double *lambdas){
11     for(int i=0;i<N+2;i++){
12         aW[i]=(i==0)?0:lambda_arm(lambdas[i],lambdas[i-1]);
13         aE[i]=(i==N+1)?0:lambda_arm(lambdas[i],lambdas[i+1]);
14         aP[i]=aW[i]+aE[i];
15     }
16 }
17
18 void GaussSeidel(int N, double *aW, double *aP, double *aE,
19                 double *T_c, double *T_s, double *max_error){
20     double error, Merror=0;
21     T_c[0]=T_s[0]; //mantenir condicions de contorn
22     T_c[N+1]=T_s[N+1];
23     for(int i=1;i<N+1;i++){ //nodes interns
24         T_c[i]=(aE[i]*T_s[i+1]+aW[i]*T_c[i-1])/aP[i];

```



```

25     error=fabs(T_s[i]-T_c[i]);
26     if(error>Merror)
27         Merror = error;
28 }
29 *max_error=Merror;
30 }

```

sources\print.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include "../headers/print.h"
4
5  void print_state(double L, double xchange, double lambda1,
6                  double lambda2, double T0, double TL){
7      printf("1D HEAT CONDUCTION IN A COMPOSITE WALL\n\n")
8          "PROBLEM STATEMENT\n\n");
9      printf("A wall of total length %fm is composed of "
10          "two materials joined at x_c = %fm. The first segment "
11          "(0<x<x_c) is made of a material A with thermal conductivity "
12          "lambda_1 = %fW/mK, and the second segment (x_c<x<L) is made "
13          "of material B with thermal conductivity lambda_1 = %fW/mK.\n\n")
14          "The system is in steady-state, and heat conduction is "
15          "one-dimensional along the x-axis. The temperature at "
16          "the left end of the wall is maintained at T_0 = %fK, and the "
17          "temperature at the right end is maintained at T_L = %fK.\n\n",
18          L,xchange,lambda1,lambda2,T0,TL);
19      printf("Assuming:\n-Steady-state heat conduction\n-No internal heat "
20          "generation\n-Perfect thermal contact between the two materials "
21          "\n-Constant thermal conductivity in each material\n-One-dimensional "
22          "conduction only.\n\nDetermine the temperature distribution T(x) along "
23          "the lenght of the wall.\n");
24 }
25
26 void print_mesh(int N, double dx, double* x, double *lambdas, double *x_vc){
27     printf("\n\n1. DISCRETIZATION OF THE MESH\n\n");
28     printf("Contemplating %d control volumes:\ndx = %f \nNode positions [m] = {"N,dx);
29     for(int i=0;i<N+2;i++){
30         printf("%f",x[i]);
31         if(i<(N+1))
32             printf(", ");
33     }
34     printf("}\nFace positions [m] = {"");
35     for(int i=0;i<N+1;i++){
36         printf("%f",x_vc[i]);
37         if(i<N)
38             printf(", ");
39     }
40     printf("}\nThermal conductivities per node [W/mK] = {"");
41     for(int i=0;i<N+1;i++){
42         printf("%f",lambdas[i]);
43         if(i<N)
44             printf(", ");
45     }
46     printf("}\n");
47 }
48
49 void print_initialf(double Tstart, double T0, double TL){
50     printf("\n\n2. INITIAL MAP DEFINITION (internal nodes)\n\n");
51     printf("T_supposed[i]=T_initial=%fK\n",Tstart);
52     printf("\n\n3. BOUNDARY CONDITIONS (external nodes)\n\n");
53     printf("T[0]=T_0=%fK\n",T0);
54     printf("T[N+1]=T_L=%fK\n",TL);
55 }
56
57 void print_coeff(){
58     printf("\n\n4. COEFFICIENT CALCULATION\n\n");
59     printf("For this case:\n  aW = lambda_w\n  aE = lambda_e\n  aP=aW+aE "

```

```

60         "\n bP = 0\nbeing lambda_w and lambda_e the harmonic lambda "
61         "between the nodes W-P and P-E.\n lambda_harm = (2*lambda_i*lambda_j)"
62         "/(lambda_i+lambda_j) for equal distances dx and surfaces S.\n");
63     }
64
65     void print_solver(){
66         printf("\n\n5. GAUSS-SEIDEL SOLVER\n\n");
67         printf("T_calculated[i]=(aE[i]*T[i+1]+aW[i]*T[i-1]+bP[i])/aP[i]\n"
68             "\n\n6. Is max|T_calculated[i]-T_supposed[i]|<delta?\n\n NO: "
69             " T_s[i]=T_c[i] and back to point 5.\n YES: point 7.\n");
70     }
71
72     void print_results(double delta, int iter, int N, double *T){
73         printf("\n\n7. RESULTS\n\n");
74         printf("Contemplating delta = %e, and with %d iterations:\n",delta, iter);
75         printf("T[i] [K] = {");
76         for(int i=0;i<N+2;i++){
77             printf("%f",T[i]);
78             if(i<(N+1))
79                 printf(", ");
80         }
81         printf("}\n\n\n8. END\n\n");
82     }

```

headers\mesh.h

```

1     #ifndef MESH_H
2     #define MESH_H
3
4     void generate_mesh(double *x, double dx, double xchange,
5                       int N, double L, double *lambdas,
6                       double lambda1, double lambda2,
7                       double *x_vc);
8
9     #endif

```

headers\solver.h

```

1     #ifndef SOLVER_H
2     #define SOLVER_H
3
4     double lambda_arm(double lambda_i, double lambda_j);
5
6     void coeff(int N, double *aW, double *aP, double *aE,
7               double *lambdas);
8
9     void GaussSeidel(int N, double *aW, double *aP, double *aE,
10                    double *T_c, double *T_s, double *max_error);
11
12     #endif

```

headers\print.h

```

1     #ifndef PRINT_H
2     #define PRINT_H
3
4     void print_state(double L, double xchange, double lambda1,
5                    double lambda2, double T0, double TL);
6
7     void print_mesh(int N, double dx, double* x, double *lambdas, double *x_vc);
8
9     void print_initialf(double Tstart, double T0, double TL);
10
11     void print_coeff();
12
13     void print_solver();
14

```

```

15 void print_results(double delta, int iter, int N, double *T);
16
17 #endif

```

The results have been proved to be independent of the given initial map given ( $T_{start}$ ). They have also been shown to be regular with respect to small changes in the numerical parameters of the code (number of control volumes  $N$ , convergence criterion  $\delta$ ). In addition, the temperatures of some nodes have been verified using the analytical method.

Finally, the results  $T[i]$  obtained have been graphed as  $T(x)$  (figure 2) with the external library "gnuplot" and the following complementary linecodes at the end of "main.c".

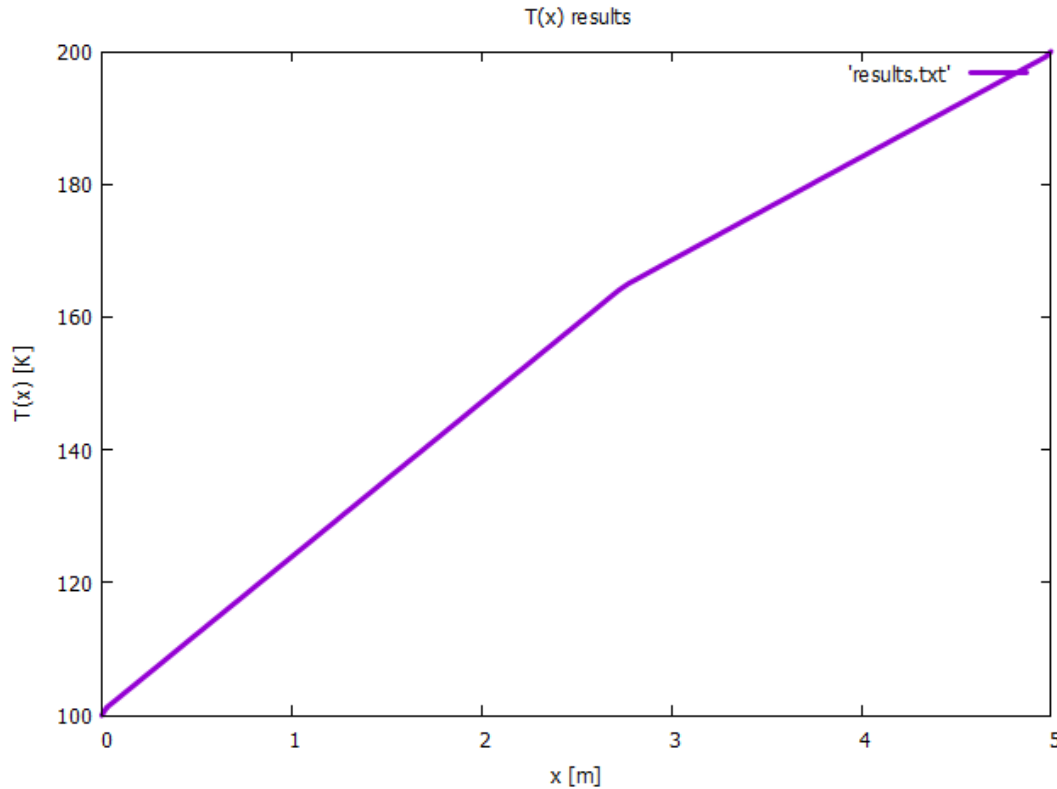


Figure 2:  $T(x)$  results

It can be seen how the graph slope changes at  $x_{change}$  due to the different thermal conductivity between the two materials as expected with the analytical resolution.

```

1  FILE *file = fopen("results.txt", "w");
2  for (int i = 0; i < N+2; i++) //escriure .txt
3      fprintf(file, "%f %f\n", x[i], T_c[i]);
4  fclose(file);
5
6  FILE *pipeForGnuplot=NULL;
7  pipeForGnuplot = popen("gnuplot -p","w"); //plot amb gnuplot
8  fprintf(pipeForGnuplot,"set title \"T(x) results\"\n");
9  fprintf(pipeForGnuplot,"set xlabel \"x [m]\"\n");
10 fprintf(pipeForGnuplot,"set ylabel \"T(x) [K]\"\n");
11 fprintf(pipeForGnuplot,"plot 'results.txt' w l linewidth 3 \n");
12 pclose(pipeForGnuplot);

```

## 5 Conclusions

In this practice, a simple one-dimensional steady-state conduction problem in a composite wall made of two materials has been successfully analyzed both analytically and numerically. The analytical solution, based on the Laplace equation and continuity conditions at the interface, provides a clear expression for the temperature distribution and the constant heat flux through the wall.

The numerical approach, implemented via the Gauss-Seidel iterative method, has reproduced the analytical results with high accuracy. The implemented C code proves to be robust and efficient, achieving convergence for very small tolerance values ( $\delta$ ) and a high number of control volumes ( $N$ ). The results are shown to be independent of the initial temperature map and stable with respect to variations in the numerical parameters.

The comparison between analytical and numerical solutions has confirmed the correctness of both methods. The plotted temperature distribution  $T(x)$  displays the expected change in slope at the interface ( $x = x_{\text{change}}$ ), corresponding to the discontinuity in thermal conductivity between materials A and B.

Overall, the study has validated the Gauss-Seidel method as a reliable numerical technique for solving steady-state heat conduction problems in composite media. It also has demonstrated the importance of mesh definition and convergence criteria for obtaining accurate and stable numerical results.